Desenvolvimento Mobile Híbrido (BSI) · PUCPR · Turma: **U**

Review Lab

Somativo 2, Catálogo Interativo 2.0 (React Native)
Catálogo de pull requests reais para estudar code review

Data de entrega: **02/11/2026**

Integrantes (grupo 10):

- Lucas Stopinski da Silva
- Lucas Bruno e Silva

1. Links

- Repositório (público): https://github.com/LucasStop/review-lab. O README tem como rodar: `yarn start` (equivale a `npx expo start`).
- Vídeo (YouTube, público): **https://youtu.be/a3q_INzumb8**

2. Tema e configuração

- **Tema:** Review Lab, um catálogo de pull requests reais de repositórios open source (padrão `expo/expo`) para estudar code review. Cada card é um PR; o detalhe mostra a descrição em texto limpo; o aluno salva PRs para estudar e marca os que estudou com uma nota de utilidade de 1 a 5.
- **API:** GitHub REST, somente leitura (`/repos/{o}/{r}/pulls`, detalhe com `Accept: application/vnd.github.html+json` e `/search/issues`). Sem token são 60 requisições por hora; o app usa cache e mostra o horário de volta quando o limite acaba.
- **Persistência:** local, AsyncStorage (favoritos, estudados com nota e colunas, por usuário).
- **Login:** local, com senha em hash SHA-256 com sal e sessão em expo-secure-store. Sem bônus de nuvem nem de auth real.

3. Autoavaliação RF01 a RF10

RF	Implementado	Arquivo principal	Observação
RF01 Catálogo	Sim	src/screens/CatalogScreen.tsx	`FlatList` com `numColumns` (2 colunas, botão para 1), `PullRequestCard` com avatar do autor e placeholder, `onEndReached`, pull-to-refresh e `keyExtractor` pelo número do PR.
RF02 Navegação	Sim	src/navigation/AppStack.tsx	native-stack; o detalhe recebe o número em `route.params` e define o título com `navigation.setOptions`.
RF03 Detalhes	Sim	src/screens/DetailScreen.tsx	Segunda requisição com axios; `body_html` convertido em blocos de texto sem tags (src/utils/sanitizeHtml.ts). A imagem é o avatar do autor, porque PR não tem imagem.
RF04 Favoritos (Context)	Sim	src/contexts/FavoritesContext.tsx	`createContext` + Provider + `useFavorites()`; a estrela do card e o botão do detalhe usam o mesmo estado.
RF05 Tela de Favoritos	Parcial	src/screens/CatalogScreen.tsx	Em vez de tela própria, o filtro “Salvos” fica na tela principal, atualiza ao desfavoritar e tem estado vazio (src/components/EmptyState.tsx).
RF06 Persistência	Sim	src/hooks/usePersistedList.ts	AsyncStorage por usuário; carga num `useEffect` e um portão no RootNavigator segura as telas até hidratar, sem lista zerada. Sem bônus de nuvem.

RF	Implementado	Arquivo principal	Observação
RF07 Login, sessão e consumidos	Sim	src/contexts/AuthContext.tsx	Login local: hash SHA-256 com sal (src/services/credentials.ts) e sessão em expo-secure-store; navegação condicional; logout. “Consumidos” são os PRs “Estudados” com nota de 1 a 5 (src/contexts/StudiedContext.tsx), listados pelo filtro “Estudados” e pelo Perfil. Sem bônus de auth real.
RF08 Busca	Sim	src/screens/CatalogScreen.tsx	`TextInput` controlado e botão “Buscar” chamam `/search/issues` (src/services/github.ts); tocar abre o detalhe; mensagem “Nenhum resultado para” o termo.
RF09 Feedback de UI	Sim	src/hooks/useFetch.ts	`loading | data | error` explícitos, `ActivityIndicator`, rodapé da página seguinte e ErrorState com “Tentar novamente”; conferido sem rede e com o limite da API esgotado. O loader em rede lenta só tem teste automatizado.
RF10 Acessibilidade	Sim	src/components/Button.tsx	`accessibilityRole`, `accessibilityLabel` e `accessibilityState` nos tocáveis, alvos de 44x44, fonte do sistema ampliada sem desligar `allowFontScaling`, contraste testado em src/theme/index.ts. VoiceOver conferido no simulador, não em aparelho físico.

Código e organização: pastas `services/`, `contexts/`, `hooks/`, `components/`, `screens/`, TypeScript, 114 testes de lógica e 174 de componente, suíte E2E com Maestro (9 de 9) e CI no GitHub. Sem segredos no repositório.

4. Flutter x React Native

Provider x Context. No Flutter o `ChangeNotifierProvider` expõe um objeto que notifica quem escuta. No RN o `createContext` + `Provider` + um hook faz o mesmo, mas o estado vive em `useState`/`useMemo` dentro do Provider e cada mudança renderiza de novo os consumidores. Em FavoritesContext.tsx o hook `useFavorites()` lança erro fora do Provider, o que o Flutter faz com o `context.read` sem Provider. O custo é que o `value` precisa de `useMemo`, senão todo consumidor renderiza sem necessidade.

GridView x FlatList. O `GridView.builder` do Flutter vira a `FlatList` com `numColumns`. Ela também só monta o que está na tela. Duas diferenças que apareceram: o `keyExtractor` precisa devolver uma chave estável, senão a lista remonta os itens (usamos o número do PR), e trocar `numColumns` na mesma lista lança erro, então o botão de 1 ou 2 colunas muda a `key` da lista em CatalogScreen.tsx. A rolagem infinita e o pull-to-refresh vêm como props (`onEndReached`, `refreshing`), sem `ScrollController`.

FutureBuilder x useEffect/useState. O `FutureBuilder` junta a requisição e os três estados num widget. No RN não há equivalente pronto: escrevemos o hook useFetch, que devolve `loading | data | error` e um `retry`, e cancela com `AbortController` quando a tela sai. É mais código, mas fica reutilizável e testável sozinho.

Hot reload x Fast Refresh / Expo Go. O hot reload do Flutter preserva o estado ao recompilar. O Fast Refresh do RN faz parecido: edita o componente e mantém o estado, mas recarrega o arquivo inteiro, perdendo o estado, quando ele exporta algo além de componentes. O Expo Go dispensa Xcode e Android Studio para rodar no aparelho, mas só acessa módulos nativos que ele já traz, e é o que impôs o expo-secure-store e o AsyncStorage em vez de bibliotecas nativas avulsas.

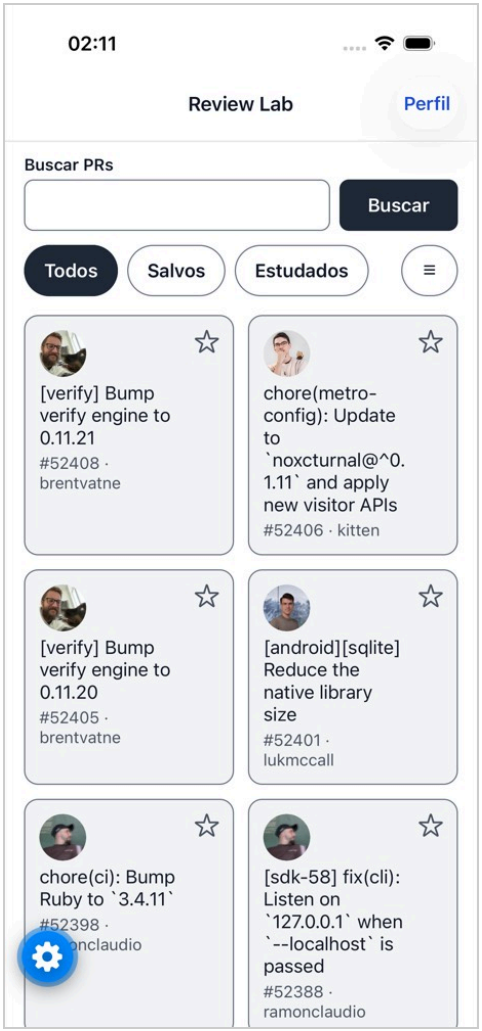
5. Explicação técnica

Tema: **navegação condicional com `AuthContext`**: o `RootNavigator` escolhe a AuthStack ou a AppStack pelo estado do contexto, e o logout só limpa a sessão, sem `navigate('Login')`. Aparece no vídeo a partir de **5:48**.

6. Screenshots



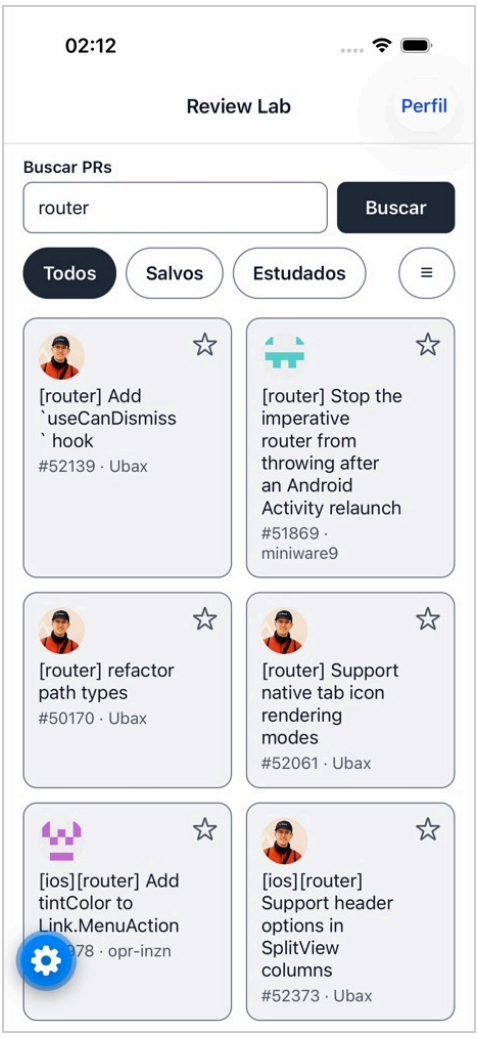
Login



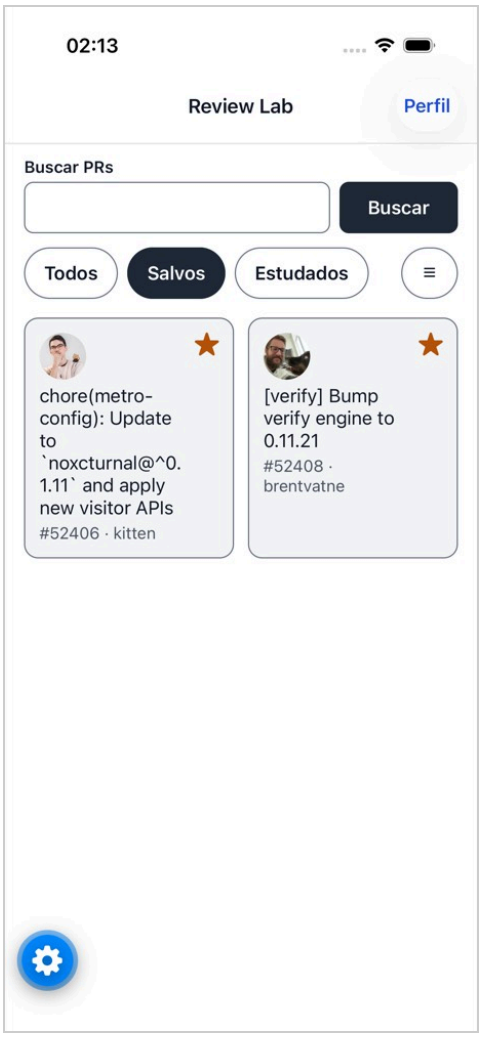
Catálogo



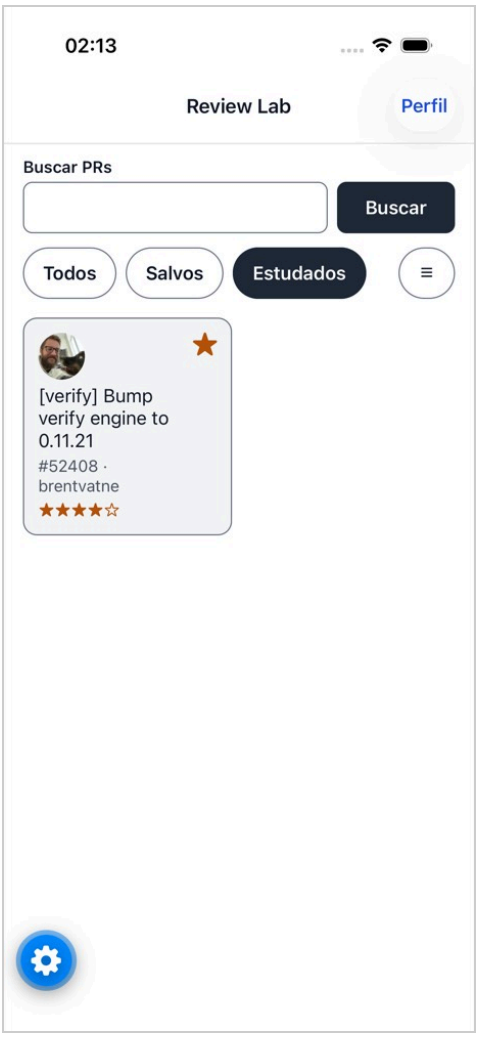
Detalhes (salvo, estudado, nota 4)



Busca



Favoritos (filtro Salvos)



Consumidos (filtro Estudados)